UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingenieria de Sistemas.

Estructura de Datos

Prof. Gilberto A. Salinas

Semana 01
Filas Secuenciales

Ciclo 2026-1

Filas secuenciales

1. Introducción archivos
2. Filas secuenciales
3. Primitivas de acceso
4. Diseño
5. Implementación
6. Resumen
7. Referencias

1. Archivos

Introducción

- La memoria principal (RAM) del computador es volátil, escasa y costosa.
- ***La memoria secundaria es permanente abundante y barata.***
  - **Magnéticos:** HD, Cintas,
  - **Electrónicos;** USB, SSD (Solid State Drive), SSHD, mSATA (Serial Advanced Tecnology Attachment), SATAe, M.2, PCIe (Peripheral Component Interconection Express), etc.
  - **Ópticos:** CD, DVD, BluyRay

1. Archivos

Caracteristicas

Es una estructura de almacenamiento secundario..

- **Persistencia.**
  - **Files:** Son permanentes, se almacenan een medio secundarios.
  - **Array:** Son volatiles, se almacean en la RAM.
- **Capacidad de almacenamiento**
  - **Files: Files:** Casi ilimitado, depende del espacio libre en el disco.
  - **Array:** Limitados por la memoria RAM disponible, puede causar error de dezboramiento.
- **Velocidad de acceso.**
  - **Files:** Son lentos, se realizan operaciones de E/S.
  - **Array:** Extramdamente rapidos. El acceso a la RAM es casi instantaneo.

1. Archivos

Caracteristicas

Es una estructura de almacenamiento secundario..

- **Formas de acceso**
  - **Files:** Generalmente secuenciales, existe el acceso aletario pero es mas complejo de implementar.
  - **Array:** Acceso directo o aleatorio, Puedes saltar a cualquier posicion a traves del indice.
- **Estrctura de datos**
  - **Files:** Son Flujo de bytes, o texto plano, no tienen estructura rigida, pueden ser CSV o JSON
  - **Array:** Son estructuras homogeneas/heterogeneas. Todos los elementos deben ser del mismo tipo y estan organizados en forma contigua

1. Archivos

Por su contenido en cada byte

- **Texto.** (plain text) Están permitidos ciertos rangos de valores en cada byte, incluyen valores hexadecimal: ox1A marca de fin de fichero. No es posible leer mas de un byte:
  - Letras, números, signos de puntuación, tabulaciones, saltos de línea, retornos de carro. (ASCII)
  - formato texto: negrita, subrayado, tamaño, etc.
- **Binarios.** Están permitidos todos los valores para cada byte. El final de file se detecta de otro modo dependiendo del SO, generalmente es la longitud del file.
  - Texto formateado, fotografías, música, archivos ejecutables

1. Archivo

Por el tipo de acceso. Es la tecnica que se utiliza para representar y almacenar registros en los archivos, veamos:

- **Secuencial**. Se accesa en forma secuencial a los elementos del archivo.
- **Directos**: Permite el acceso al registro deseado directamente mediante la formula :
  - ***direccionFisica = direccionLogica*tamanoRegistroEnBytes.***

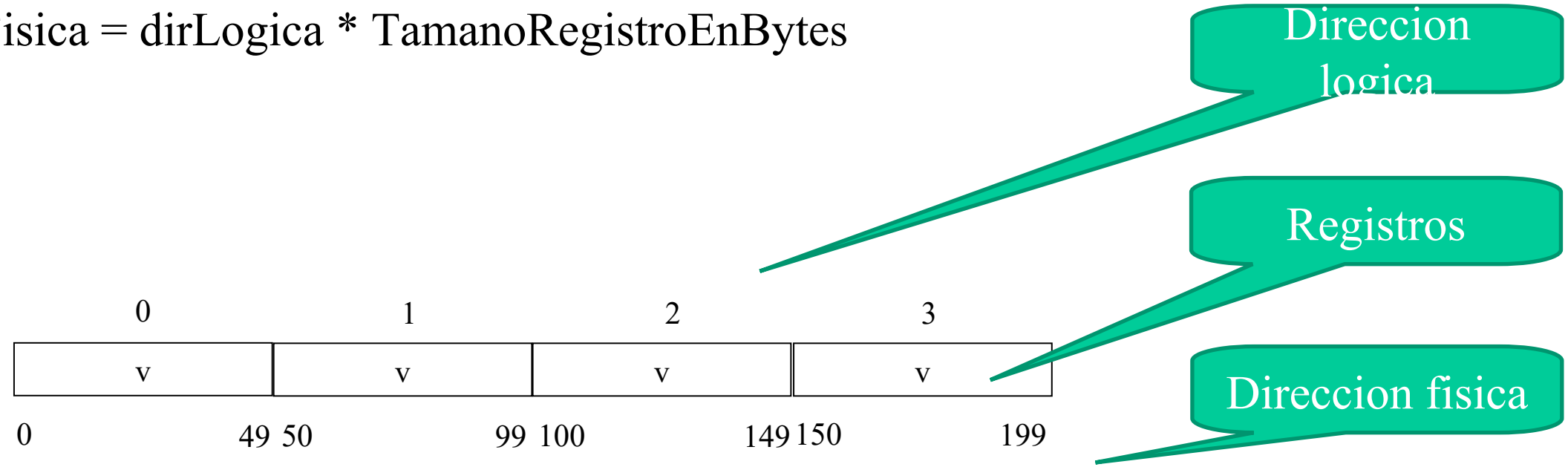
Fundamentos: Sea el registro de alumnus.

- REGISTRO ALUMNO
  - ENTERO codigo // 4 bytes
  - CARACTER nom[STR42] // 42
  - REAL nota // 4
- FIN_REGISTRO // 50 bytes

1. Archivo

- REGISTRO ALUMNO
  - ENTERO codigo // 4 bytes
  - CARACTER nom[42] // 42
  - REAL nota // 4

  50 bytes
- FIN_REGISTRO
- dirFisica = dirLogica * TamanoRegistroEnBytes



1. Archivos

Por dirección del flujo de datos:

- **Archivo de entrada.** Los datos se leen por el programa desde el file.
- **Archivo de salida.** Los datos se ponen o escriben por el programa hacia el file.
- **Archivo de entrada/Salida.** Los datos pueden ser leídos o escritos por el programa.

```
REGISTRO Practicas
    ENTERO codAlu
    REAL  p[MAX]
FIN_REGISTRO

REGISTRO PromPracticas
    ENTERO codAlu
    REAL  prom
FIN_REGISTRO
```

```
REGISTRO Calific
    ENTERO codAlu
    REAL  ep
    REAL  ef
    REAL  pp
    REAL  nf
FIN_REGISTRO
```

```
REGISTRO ALUMNO
    ENTERO codAlu
    CARACTER nomAlu[STR30]
     ENTERO scola
    CARACTER dir[STR30]
    CARACTER dist[STR30]
    CARACTER correo[STR30]
    CARACTER fono[11]
FIN_REGISTRO
```

filePracticas.txt
fileCalific.txt

filePromPracticas.txt
fileCalific.txt

1. Archivos

Tipos de archivos por su function:

•**Maestro**. Son archivos perennes

•**Transaccion**. Auxiliaries o temporales, se eliminan

•**Reporte**. Se crean para reports, generalmente son temporales

Modos de apertura de archivos en lenguaje C. Existen tres modos basicos de apertura, vease vuestro libro de texto para las otras variantes:

•“r” Abre un archivo de texto para lectura

•“w” Abre un file de texto para escritura, si no existe lo crear y si existe lo destruye.

•“a” Abre un archivo de texto para agregar al final del file.

1. Archivos

Ventajas

- **Examinar**
- **Compartir**
- **Persistencia.** Nos permite almacenar datos en el tiempo y espacio.
- **Capacidad.** Siempre serán las unidades de mayor capacidad

Desventajas:

- **No centralizada**
- **Velocidad.** Siempre estará en desventaja con la velocidad de acceso a la RAM
- **Acceso.** Cuando un file es accedido por varios usuarios de forma simultanea puede presentar problemas de concurrencia.
- **Files compartidos.** Requiere protección.

2. Fila Secuencial

Definición

- Es una estructura de datos para almacenamiento secundario.
- Conjunto relacionado y organizado de información almacenados en un dispositivo de almacenamiento secundario.
- Características:
  - Para accesar a los elementos siempre se hace desde el inicio. Para llegar al e-enésimo elemento se necesita haber recorrido los n-1 elementos.
  - La inserción de nuevo elementos siempre se hacen al final de la fila.
  - Solo se pueden abrir para lectura o escritura rara vez para los dos modos.

2. Primitivas de acceso

Son operaciones elementales de las filas que nos permiten accesar ha ellas y son:

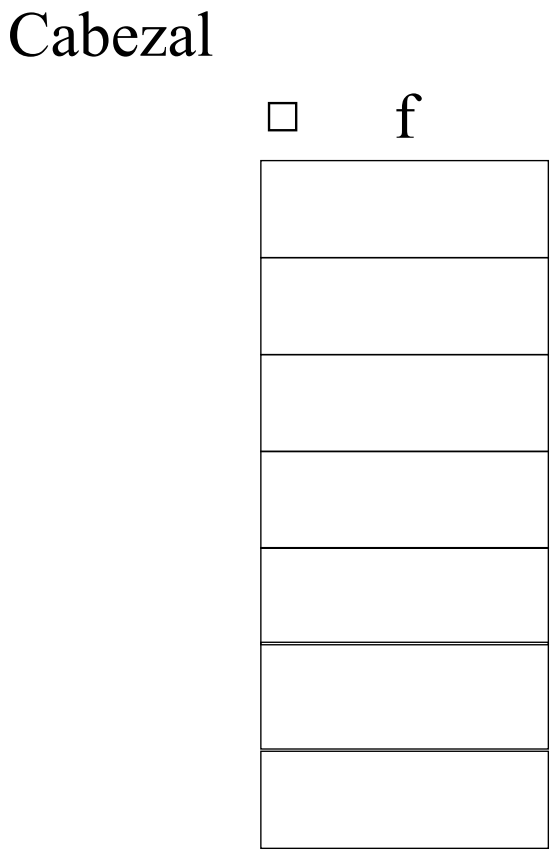
Sea **f** una variable de tipo archivo, y **v** una variable de tipo simple, compuesta o estructurado.

- ABRIR (FILE f)
- PONER(FILE f, TD v)
- MARCAR(FILE f, TD ¬)
- TOMAR(FILE f, TD v)
- ULTIMO(FILE f).

3. Primitivas de acceso

Sea **f** una variable de tipo archivo, y **v** una variable de tipo simple o compuesta.

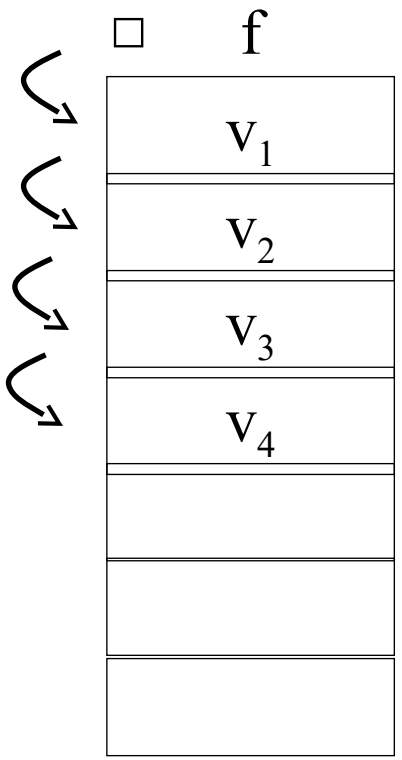
- ABRIR (FILE, f)
  - El cabezal de lectura se posiciona antes del primer elemento de la fila.



3. Primitivas de acceso

Sea **f** una variable de tipo archivo, y **v** una variable de tipo simple o compuesta.

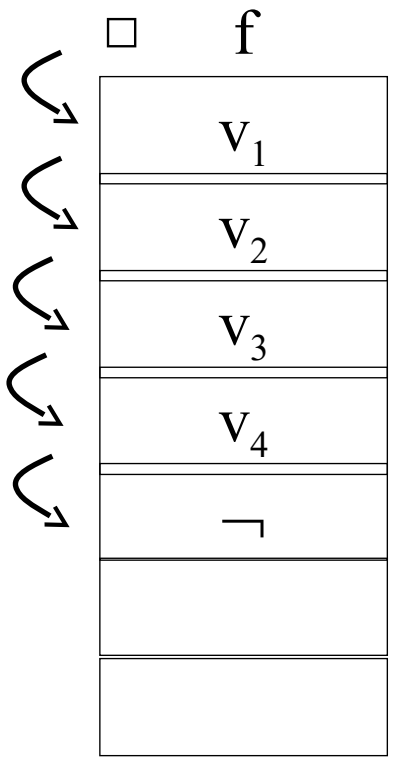
- PONER (FILE f, TD v)
  - El cabezal de lectura avanza una posicion y se pone el valor de v.
    - El cabezal de lectura avanza una posicion.
    - Se pone, escribe, graba o se salva el valor de v.
    - Este sera el ultimo elemento de la fila. □



3. Primitivas de acceso

Sea **f** una variable de tipo archivo, ¬ la marca de fin de file.

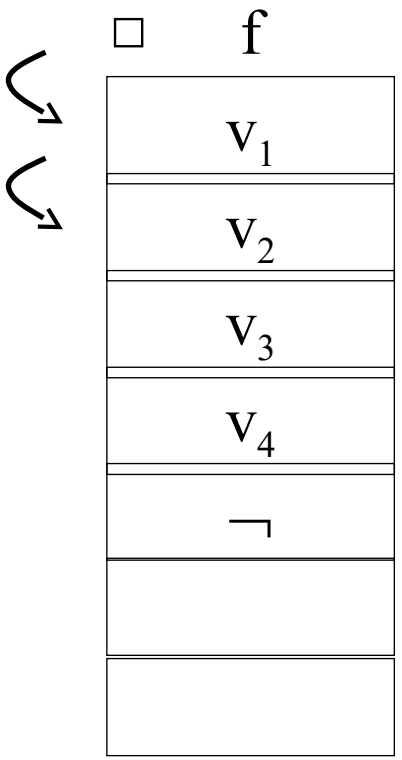
- MARCAR (FILE f, TD ¬)
  - El cabezal de lectura avanza una posicion y se pone la marca de fin file.
    - El cabezal de lectura avanza una posicion.
    - Se pone, escribe o graba la marca de fin de file.
    - La fila queda cerrada o “sellada”.



3. Primitivas de acceso

Sea **f** una variable de tipo archivo, **v** una variable de tipo simple o compuesta.

- TOMAR (FILE f, TD v)
  - El cabezal de lectura avanza una posicion y se toma el valor de esa posicion.
    - El cabezal de lectura avanza una posicion.
    - Se toma, lee o recupera el valor de esa posicion.
    - El valor recuperado se asigna a la variable v.

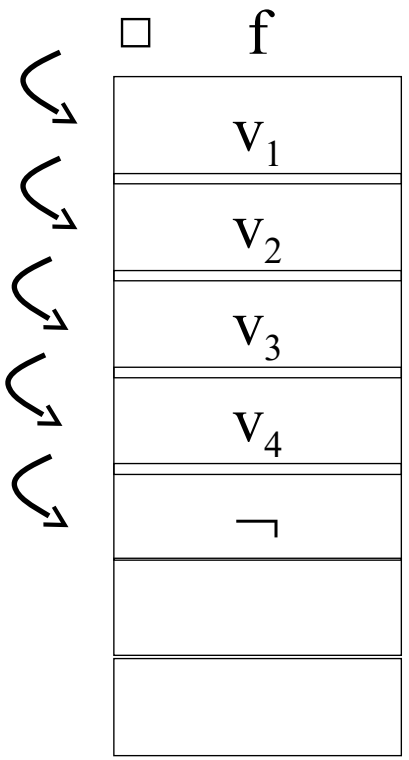


3. Primitivas de acceso

Sea **f** una variable de tipo archivo.

•ULTIMO (FILE f)

– Es un predicado que devuelve VERDAD si encuentra la marca de fin de fila y FALSO en otro caso.

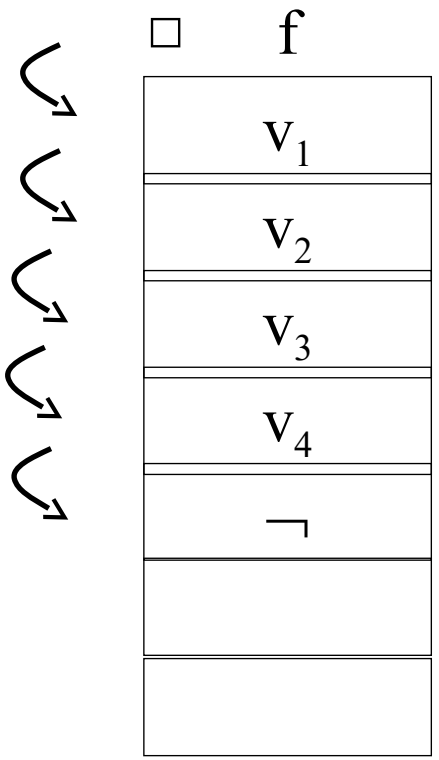


3. Ejemplo

Implemente un subprograma para crear una fila secuencial f de numeros enteros.

Solucion:

- ACCION crearFile( )
  - FILE f
  - ABRIR (f)
  - ENTERO v, i ← 0, n
  - LEER(n)
  - MIENTRAS (i < n)
    - LEER(v)
    - PONER(f, v)
    - i ← i + 1
  - FIN_MIENTRAS
  - MARCAR (f, ¬)
- FIN_ACCION

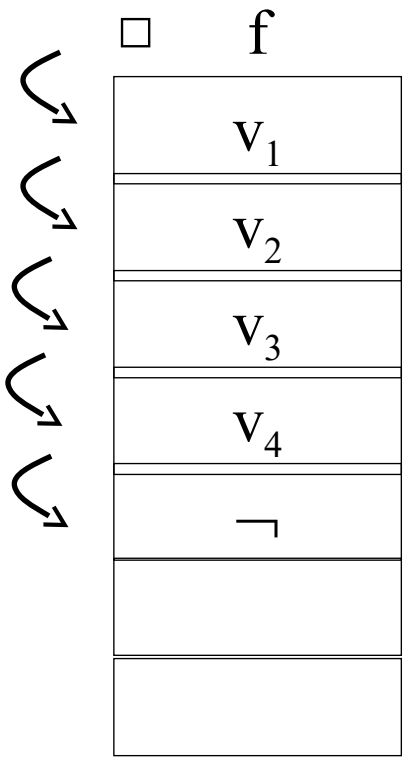


3. Ejemplo

Implemente un subprograma para mostrar los elementos de la fila secuencial g de numeros enteros.

Solucion:

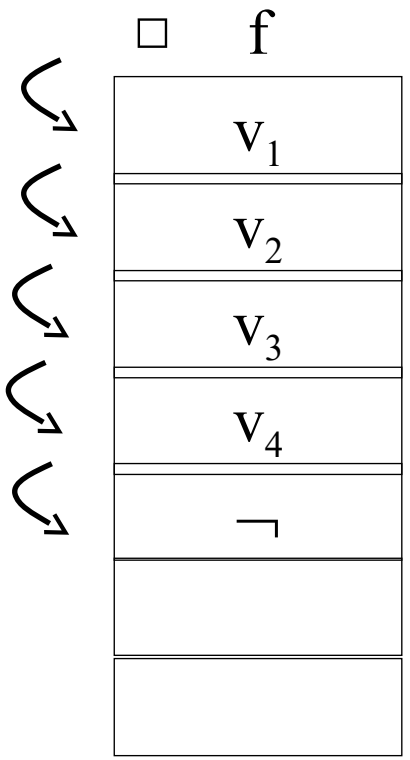
- ACCION mostrarFile( )
  - FILE g
  - ABRIR (g)
  - ENTERO v
  - TOMAR(g, v)
  - MIENTRAS (NO ULTIMO(g))
    - ESCRIBIR(v)
    - TOMAR(g, v)
  - FIN_MIENTRAS
  - MARCAR (g, ¬)
- FIN_MIENTRAS



3. Ejemplo

- ACCION crearFile( )
  - FILE f
  - ABRIR (f)
  - ENTERO v, i ← 0, n
  - LEER(n)
  - MIENTRAS (i < n)
    - LEER(v)
    - PONER(f, v)
    - i ← i + 1
  - FIN_MIENTRAS
  - MARCAR (f, ¬)
- FIN_MIENTRAS

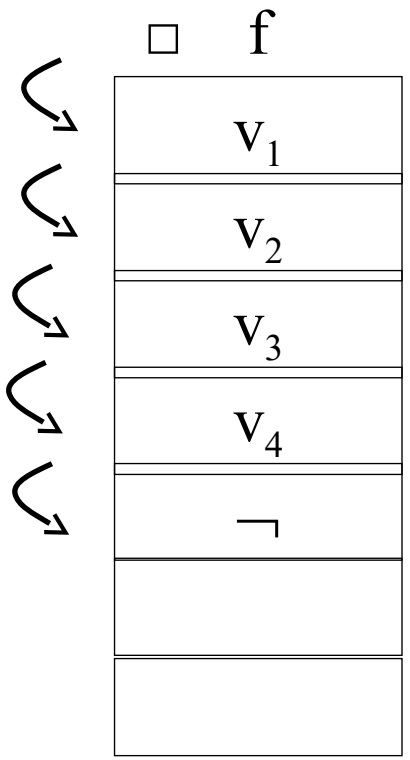
- int main(){
  - FILE *f;
  - F = fopen(“fentero.txt”, “a”);
  - if(f !=NULL){
    - printf(“No se pudoabrir el file…\n”);
    - exit(0);
  - }
  - int v, i =0, n;
  - scanf(“%d”,&n);
  - while (i < n){
    - scanf(“%d”,&v);
    - fwrite(&v, sizeof(v), 1,f);
    - i ← i + 1;
  - }
  - fclose (f);
- }



3. Ejemplo

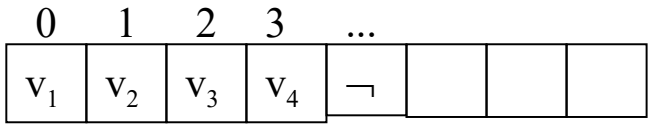
- ACCION mostrarFile( )
  - FILE g
  - ABRIR (g)
  - ENTERO v
  - TOMAR(g, v)
  - MIENTRAS (NO ULTIMO(f))
    - ESCRIBIR(v)
    - TOMAR(g, v)
  - FIN_MIENTRAS
  - MARCAR (g, ¬)
- FIN_MIENTRAS

- int main(){
  - FILE *g;
  - int v;
  - g = fopen("fsentero.txt", "r");
  - if(g !=NULL){
    - printf("No se pudo abrir el file…\n");
    - exit(0);
  - }
  - fread(&v, sizeof(v), 1,g);
  - while (!feof(g)){
    - printf("%d",v);
    - fread(&v, sizeof(v), 1,g);
  - }
  - fclose (g);
- }



3. TAD Fila

TAD Fila [tipo ENTERO]		
Objeto abstracto	$f \rightarrow$	



{Invariante: $v_i \in Z$}		
crearFila	Fila x ENTERO	Fila
mostrarFila	Fila	ENTERO
buscarDatoFila	Fila	ENTERO
eliminarDatoFila	Fila x ENTERO	Fila
editarDatoFila	Fila x ENTERO	Fila

ACCION crearFila ( )

//Crea una fila secuencial de enteros

{pre: $f \in$ FILE, $f\ \exists \wedge v \in Z$}

{pos: f no existe $\vee$ f de enteros}

3. TAD Fila

TAD Fila [tipo ENTERO]		
Objeto abstracto	$f \rightarrow$	

0	1	2	3	...			
v_1	v_2	v_3	v_4	¬			

{Invariante: $v_i \in Z$}

crearFila	Fila x ENTERO	Fila
mostrarFila	Fila	ENTERO
buscarDatoFila	Fila	ENTERO
eliminarDatoFila	Fila x ENTERO	Fila
editarDatoFila	Fila x ENTERO	Fila

ACCION mostrarFila (Fila f)

//Muestra los elementos de la fila

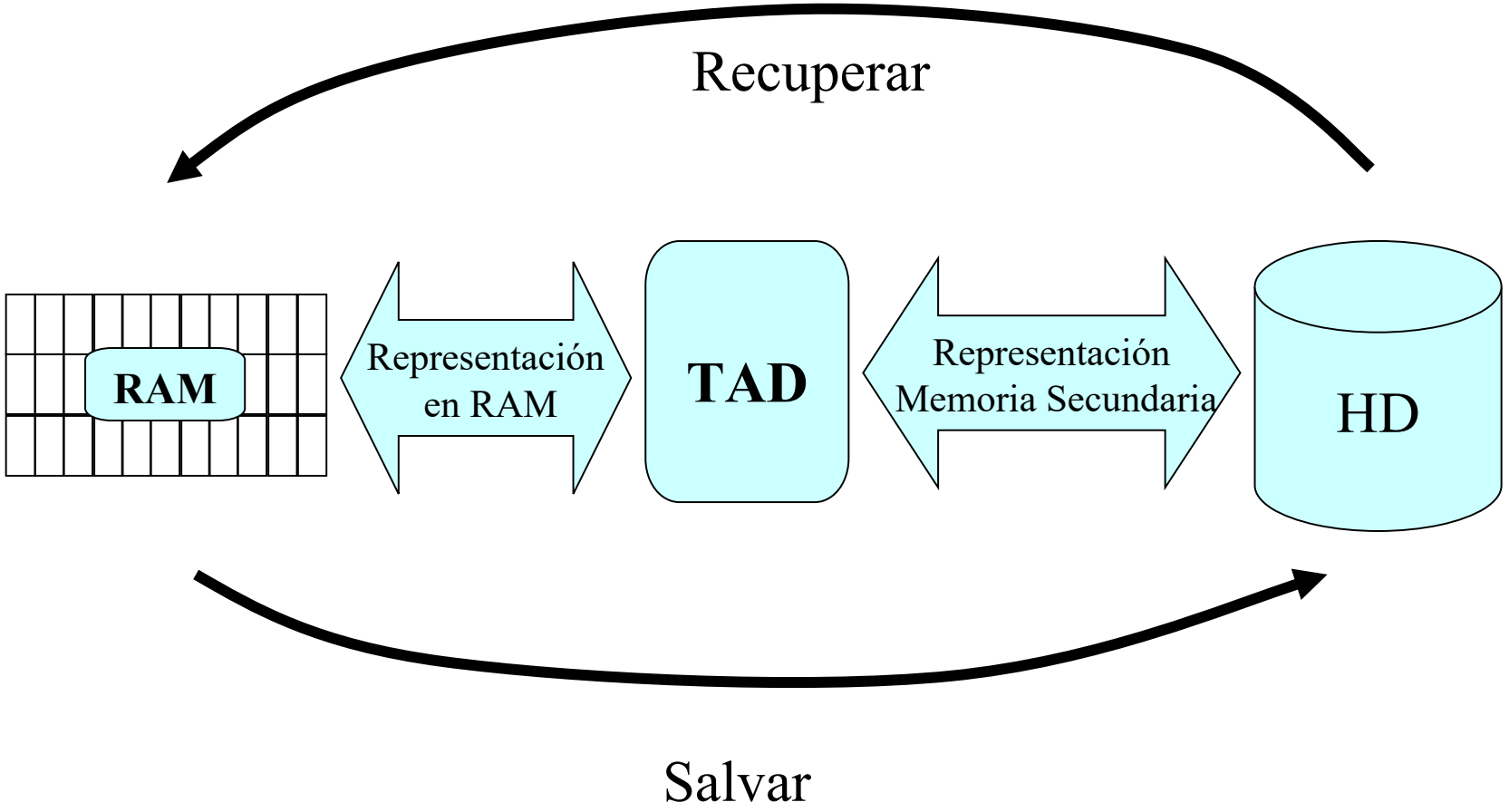
{pre: f= < > ∨ f = <e1,e2,…, en>}

{pos: fila esta vacia ∨ e1, e2, …, en}

¿Corresponde a la solucion del problema?

5. Implementación

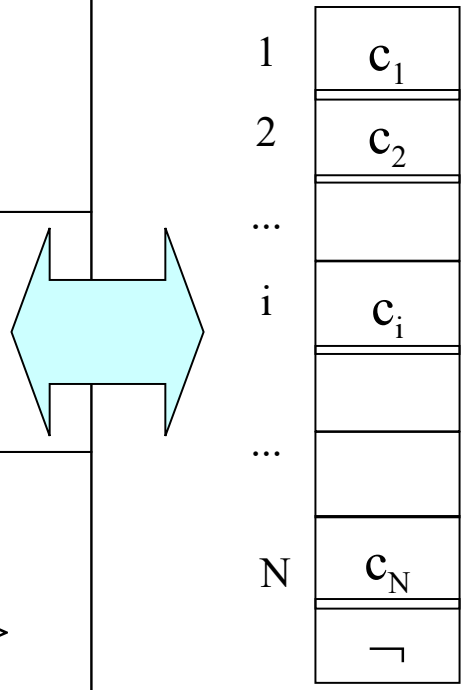
Persistencia de un TAD: Define la manera como se almacena y se recupera de la memoria secundaria (HD) la información asociada a un ***Objeto abstracto.***



4. Implementación

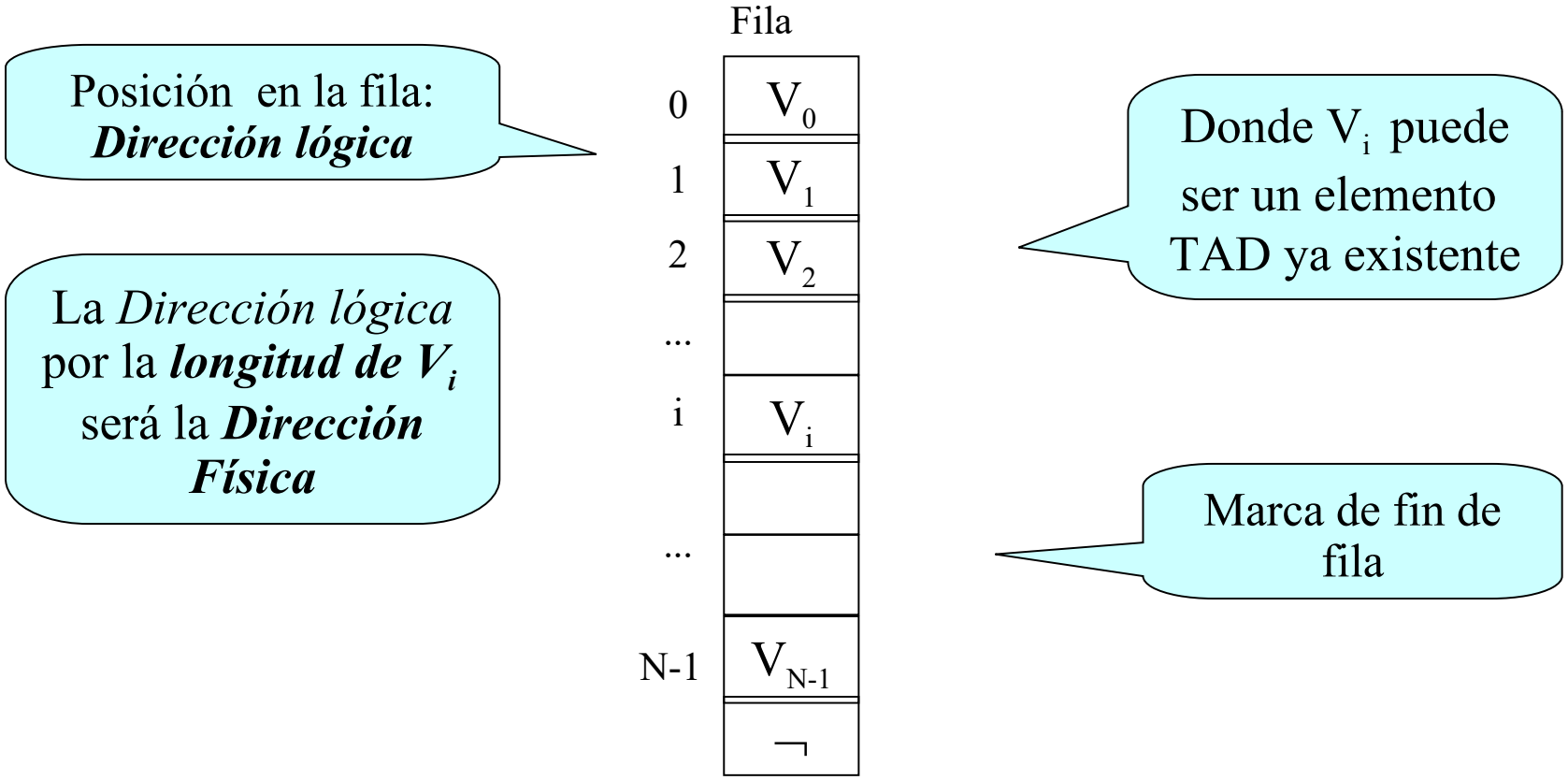
Persistencia: Cada objeto trata de sobrevivir, esto es, la información y solo con esto debe tratarse de reconstruir la representación.

Representación	Formalismo	Esquema Persistencia
c_1 c_2 c_n \0	"$c_1\ c_2\ \ldots c_N$"	
c_1 c_2 c_n	$\{c_1, c_2, \ldots, c_N\}$	
c_1 → c_2 → c_n *	$< c_1, c_2, \ldots, c_N >$	



5. Implementación

Estructura de Datos para el TAD Fila secuencial: Representación en memoria secundaria



6. Resumen

- FS. Es una estructura para almacenamiento secundario.
- Primitivas de acceso. Son las operaciones básicas de acceso a la FS.
- En la representación física de los registros podemos deducir porque se denomina FS
- En un archivo solo podemos adicionar registros, la inserción y eliminación son muy costosas.
- Diseno. Son los conceptos que permiten al ingeniero diseñar y especificar antes de construir.
- Recordar lo que se almacena en FS es la información y no la estructura.
- El proceso de la información (cambio de estado del objeto) se da en la memoria principal.

Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Villalobos J. (1996) Diseño y Manejo de Estructuras de Datos en C. Santa Fe de Bogotoam Colombia:McGraw-Hill.
- Ficheros en C y C++ .[Ultima actualización 13.06.2017] [Accesaro en Lima, 19.05.2020, 1820 horas], http://c.conclase.net/ficheros/
- Diseno.[Ultima actualización 07.11.2015] [Accesaro en Lima, 18.03.2020, 1820 horas], https://es.wikipedia.org/wiki/Dise%C3%B1o
- Ingenieria.[Ultima actualización 24.02.2014] [Accesaro en Lima, 18.03.2020, 1910 horas], https://es.wikipedia.org/wiki/Ingenier%C3%ADa